Products Pricing Documentation Community

npm

Sign Up Sign In

Search packages Search

Miss any of our Open RFC calls? Watch the recordings here! »

# next-connect

0.9.1 • Public • Published a month ago

Readme

Explore BETA

1 Dependency

4 Dependents

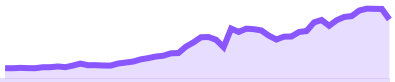
22 Versions

Install

```
> npm i next-connect
```

Weekly Downloads

8,667



| Version | License |
| --- | --- |
| 0.9.1 | MIT |
| **Unpacked Size** | **Total Files** |
| 16.5 kB | 8 |
| **Issues** | **Pull Requests** |
| 5 | 0 |

Homepage

github.com/hoangvvo/next-connect#readme

Repository

github.com/hoangvvo/next-connect

Last publish

a month ago

Collaborators

Try on RunKit

Report malware

# next-connect

npm v0.9.1 | PASSED | codecov 100% | minified size 3.3 KB | downloads 221.97K/year | PRs welcome

The smol method routing and middleware for **Next.js** (also works in **micro** or **Node.js HTTP Server**). Powered by **trouter**.

## Installation

```
npm install next-connect
// or
yarn add next-connect
```

## Usage

`next-connect` is often used in **API Routes**:

```
// pages/api/index.js
```

```
import nc from 'next-connect';

const handler = nc()
  .use(someMiddleware())
  .get((req, res) => {
    res.send('Hello world');
  })
  .post((req, res) => {
    res.json({ hello: 'world' });
  })
  .put(async (req, res) => {
    res.end('async/await is also supported!');
  })
  .patch(async (req, res) => {
    throw new Error('Throws me around! Error can be caught anc
  });

export default handler;
```

For usage in pages with **`getServerSideProps`** , see **`.run`** .

See an example in **nextjs-mongodb-app** (CRUD, Authentication with Passport, and more)

### TypeScript

By default, the base interfaces of `req` and `res` are `IncomingMessage` and `ServerResponse` . When using in API Routes, you would set them to `NextApiRequest` and `NextApiResponse` by providing the generics to the factory function like so:

```
import { NextApiRequest, NextApiResponse } from 'next';
import nc from 'next-connect';

const handler = nc<NextApiRequest, NextApiResponse>()
```

In each handler, you can also define custom properties to `req` and `res` (such as `req.user` or `res.cookie` ) like so:

```
interface ExtendedRequest { user: string };
interface ExtendedResponse { cookie: (name: string, value: st

handler.post<ExtendedRequest, ExtendedResponse>((req, res) =>
  req.user = 'Anakin';
  res.cookie('sid', '8108');
})
```

# API

The API is similar to **Express.js** with several differences:

- It does not include any helper methods or template engine (you can incorporate them using middleware)
- It does not suppoprt error-handling middleware pattern. Use `options.onError` instead.

It is more like good ol' **connect** (hence the name) with method routing.

### nc(options)

Initialize an instance of `next-connect`.

#### options.onError

Accepts a function as a catch-all error handler; executed whenever a middleware throws an error. By default, it responses with status code `500` and error message if any.

```
function onError(err, req, res, next) {
  logger.log(err);

  res.status(500).end(err.toString());
  // OR: you may want to continue
  next()
}

const handler = nc({ onError });

handler
```

```
  .use((req, res, next) => {
    throw new Error('oh no!');
    // or use next
    next(Error('oh no'));
  })
  .use((req, res) => {
    // this will run if next() is called in onError
    res.end('error no more');
  });
```

### options.onNoMatch

Accepts a function of `(req, res)` as a handler when no route is matched. By default, it responses with `404` status and `not found` body.

```
function onNoMatch(req, res) {
  res.status(404).end('page is not found... or is it')
}

const handler = nc({ onNoMatch });
```

### .use(base, ...fn)

`base` (optional) - match all route to the right of `base` or match all if omitted.

`fn` (s) are functions of `(req, res[, next])` **or** an instance of `next-connect`, where it will act as a sub application.

```
// Mount a middleware function
handler.use((req, res, next) => {
  req.hello = 'world';
  // call next if you want to proceed to next in chain
  next();
});
// Or include a base
handler.use('/foo', fn); // Only run in /foo/**

// Mount an instance of next-connect
```

```
const common = nc().use(midd1).use(midd2); // You may have so
const auth = nc().use('/dashboard', checkAuth);
const subapp = nc().get(getHandle).post('/baz', postHandle);
handler
  .use(common) // `midd1` and `midd2` runs everywhere
  .use(auth) // `checkAuth` runs on /dashboard/*
  .use('/foo', subapp); // `getHandle` runs on /foo while `po

// You can use a library too.
handler.use(passport.initialize());
```

### .METHOD(pattern, ...fns)

`METHOD` is a HTTP method ( `GET` , `HEAD` , `POST` , `PUT` , `PATCH` , `DELETE` , `OPTIONS` , `TRACE` ) in lowercase.

`pattern` (optional) - match all route based on **supported** pattern or match all if omitted.

`fn` (s) are functions of `(req, res[, next])` . This is ideal to be used in API Routes.

```
handler.use('/user', passport.initialize());
handler.get('/user', (req, res, next) => {
  res.json(req.user);
});
handler.post('/users', (req, res, next) => {
  res.end('User created');
});
handler.put('/user/:id', (req, res, next) => {
  // https://nextjs.org/docs/routing/dynamic-routes
  res.end(`User ${req.query.id} updated`);
});
handler.get((req, res, next) => {
  res.end('This matches whatever route')
})
```

However, since Next.js already handles routing (including dynamic routes), we often omit `pattern` in `.METHOD`.

### .all(pattern, ...fns)

Same as **.METHOD** but accepts *any* methods.

### .run(req, res)

Runs `req` and `res` the middleware and returns a **promise**. It will **not** render `404` on not found or `onError` on error.

This can be useful in **`getServerSideProps`**.

```
// page/index.js
export async function getServerSideProps({ req, res }) {
  const handler = nc()
    .use(passport.initialize())
    .post(postMiddleware);
  try {
    await handler.run(req, res);
  } catch (e) {
    // handle the error
  }
  // do something with the upgraded req and res
  return {
    props: { user: req.user }
  };
}
```

## Using in other frameworks

`next-connect` supports any frameworks that has the signature of `(req, res)`.

### Micro

```
const {send} = require('micro')
const handler = require('next-connect')()
```

```
handler
  .use(middleware)
  .get((req, res) => {
    res.end('Hello World!')
  })
  .post((req, res) => {
    send(res, 200, { hello: 'world' })
  });

module.exports = handler;
```

### Node.js HTTP Server

```
const http = require('http')
const handler = require('next-connect')()

handler
  .use(middleware)
  .get((req, res) => {
    res.end('Hello world');
  })
  .post((req, res) => {
    res.setHeader('content-type', 'application/json');
    res.end(JSON.stringify({ hello: 'world' }));
  });

http.createServer(handler).listen(PORT);
```

## Contributing

Please see my **contributing.md**.

## License

**MIT**

## Keywords

javascript nextjs middleware router connect


### Support

Help

Community

Advisories

Status

Contact npm

### Company

About

Blog

Press

### Terms & Policies

Policies

Terms of Use

Code of Conduct

Privacy